

COMPTE RENDU DE PROJET : SYSTÈMES RÉPARTIS & CALCUL PARALLÈLE

1. Introduction & Objectifs

Dans le cadre du module **Systèmes répartis / calcul parallèle**, ce projet s'intéresse à l'évaluation des performances de calcul entre une architecture multi-cœurs centrale (**CPU**) et une architecture massivement parallèle graphique (**GPU**).

L'application choisie relève du domaine des mathématiques financières : le pricing d'une option européenne de type *Call* à l'aide de la **méthode de Monte Carlo**. L'objectif est double :

1. Valider la précision de la convergence de la simulation statistique par rapport à la formule analytique exacte de Black-Scholes.
2. Analyser le gain en temps d'exécution (**Speedup**) fourni par l'exploitation d'une carte graphique (NVIDIA T4/L4 sous environnement CUDA) via la bibliothèque **CuPy**, comparé à une approche CPU parallélisée avec le module **multiprocessing** de Python.

2. Contexte Théorique & Financier

2.1. Option Européenne de type Call

Un Call européen confère à son détenteur le droit (et non l'obligation) d'acheter un actif sous-jacent à une date future déterminée T (maturité) à un prix fixé aujourd'hui K (prix d'exercice ou *strike*). Le gain d'un tel contrat à l'échéance, appelé *payoff*, est défini par la relation :

$$\text{payoff} = \max(S_T - K, 0)$$

où S_T représente la valeur de l'actif à la maturité T .

2.2. Dynamique du sous-jacent (Modèle Black-Scholes)

Sous l'hypothèse d'un modèle de Black-Scholes, le cours de l'action suit un mouvement brownien géométrique. Le prix futur S_T peut être simulé directement à l'aide de l'équation différentielle stochastique résolue :

$$S_T = S_0 \times \exp\left(\left(r - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}Z\right)$$

Avec les paramètres suivants définis dans notre étude :

- $S_0 = 100.0$: Prix initial de l'action
- $K = 100.0$: Prix d'exercice (*At-the-money*)
- $T = 1.0$: Échéance du contrat (1 an)
- $r = 0.05$: Taux d'intérêt sans risque (5 %)
- $\sigma = 0.20$: Volatilité annuelle du sous-jacent (20 %)
- $Z \sim \mathcal{N}(0,1)$: Une variable aléatoire suivant une loi normale standard.

2.3. Algorithme de Monte Carlo

La méthode consiste à générer un grand nombre N de trajectoires indépendantes pour S_T en échantillonnant la variable Z . Le prix de l'option estimé correspond à la moyenne empirique actualisée des gains simulés :

$$C \approx e^{-rT} \times \frac{1}{N} \sum_{i=1}^N \max(S_T^{(i)} - K, 0)$$

Cette méthode est dite « *trivialement parallèle* » (ou *embarrassingly parallel*) puisque chaque simulation est strictement indépendante des autres, rendant ce problème idéal pour le calcul distribué et vectoriel.

3. Architecture Technique & Implémentation

3.1. Structure du Projet (Normes Git)

Pour garantir la reproductibilité et la propreté du code, l'arborescence suivante a été générée de manière automatisée à la racine de l'espace de travail :

- `src/monte_carlo.py` : Module Python centralisé hébergeant les fonctions de calcul.

- notebooks/ : Notebook d'expérimentation pas-à-pas.
- results/ : Exportation des logs et du fichier de données brutes benchmark_results.csv.
- report/figures/ : Emplacement de sauvegarde des visualisations graphiques à destination du rapport.
- requirements.txt & .gitignore : Fichiers de gestion d'environnement (dépendances et exclusion des fichiers temporaires LaTeX/Python).

3.2. Stratégie de parallélisation CPU (multiprocessing)

Sur CPU, l'utilisation native d'un unique cœur saturerait rapidement la mémoire et le temps de calcul lors de fortes simulations. L'implémentation utilise un modèle de découpage en paquets (*chunks*) à l'aide d'un Pool de processus (contexte fork).

Pour un nombre de simulations N et P processeurs disponibles, la tâche est divisée en P sous-tâches indépendantes générant chacune N/P variables aléatoires (avec des graines de génération aléatoires distinctes *seed* pour éviter la corrélation). Chaque processus renvoie sa somme partielle, agrégée à la fin sur le processus père.

3.3. Stratégie de parallélisation GPU (CuPy)

La bibliothèque CuPy a été retenue car elle fournit un accès direct aux primitives CUDA de NVIDIA tout en conservant une syntaxe identique à NumPy. Contrairement au CPU qui traite les données de manière séquentielle ou sur un nombre limité de threads, le GPU effectue des calculs vectoriels massifs en allouant directement les tableaux dans sa mémoire globale (VRAM).

Afin de mesurer de manière rigoureuse le temps d'exécution réel (en excluant le caractère asynchrone des lancements de noyaux CUDA), des barrières de synchronisation explicites ont été injectées avant et après la capture du temps :

Python

```
cp.cuda.Stream.null.synchronize()
```

De plus, une gestion de la mémoire a été mise en place via `cp.get_default_memory_pool().free_all_blocks()` pour éviter les fuites de mémoire sur les simulations à grande échelle ($N = 10^7$).

4. Analyse des Résultats Expérimentaux

Les tests de performance ont été menés sur quatre échelles distinctes : 10 000, 100 000, 1 000 000, et 10 000 000 simulations. La valeur de référence est le **prix théorique analytique** calculé via la fonction de répartition de la loi normale (formule de Black-Scholes), qui s'élève à : **10.450586**.

4.1. Convergence et Précision Statistique

Comme illustré par l'évolution de l'erreur absolue, la précision des deux versions (CPU et GPU) s'améliore à mesure que N augmente, conformément à la **Loi Forte des Grands Nombres** et au théorème central limite (convergence en $\mathcal{O}(1/\sqrt{N})$). Les prix calculés convergent fidèlement vers la valeur théorique de **10.45**, validant la justesse mathématique des implémentations vectorielles.

4.2. Analyse du Temps d'Exécution et du Speedup

D'après les analyses de profils de calculs effectuées, le comportement des deux architectures met en évidence plusieurs phases :

1. Faibles volumétries ($N = 10\,000$) :

Le CPU affiche de bonnes performances opérationnelles tandis que le GPU pâtit de l'**overhead** d'initialisation (latence liée au transfert des instructions, allocation mémoire sur la carte graphique et temps de latence du premier appel CUDA). À ce stade, le Speedup est au plus bas, rendant l'usage du GPU peu avantageux pour de petites tâches.

2. Fortes volumétries ($N = 10\,000\,000$) :

Lorsque la charge de calcul s'intensifie, le CPU subit la surcharge liée à la gestion de ses processus lourds et aux limites physiques de ses cœurs. À l'inverse, le GPU atteint son plein rendement. Les millions de simulations sont traitées en parallèle par les milliers de cœurs CUDA (Architecture matérielle sous-jacente de type Tesla T4 ou L4), ce qui permet d'obtenir le **meilleur speedup du benchmark**.

5. Conclusion

Ce projet a permis de mettre concrètement en évidence l'apport du calcul sur processeur graphique pour des simulations statistiques de type Monte Carlo. Bien que le CPU parallélisé reste compétitif sur de petits échantillons, le passage à l'échelle démontre de façon indiscutable la supériorité du GPU avec un facteur d'accélération majeur (Speedup maximum atteint à $N = 10^7$).

Le dépôt généré respecte l'ensemble des standards de l'ingénierie logicielle (architecture modulaire, documentation claire et configuration Git complète), prêt à être déployé sur une plateforme de gestion de version telle que GitHub.